

DemonKing 기술문서
ARogueGameModeBase 편



작성자: 곽민규

- 순서

1. 목적.

2. 헤더.

3. Cpp동작 설명.

4. 테스트방법.

● 목적

ARogueGameModeBase는 런(한 판)의 진행 흐름을 서버 권위로 제어하기 위한 GameMode입니다.

런 시작 : StartRun()

스테이지 진행: AdvanceStage()

런 종료: EndRun(Reason)

스테이지에 따라 맵 이동: ServerTravelToStage(StageId)

현재는 스테이지 3개를 굴리되, 맵은 L_Run 하나로 고정해서 테스트가 가능하도록 만든 뼈대입니다.

● 헤더 구조 설명

#include 구성

```
#include "DemonKing/GameFlow/RogueRunTypes.h"
```

enum클래스를 사용하기 위해서 헤더 파일에 해당 include 경로를 배치함.

Exec 함수 3개

```
UFUNCTION(Exec)
void StartRun();

UFUNCTION(Exec)
void AdvanceStage();

UFUNCTION(Exec)
void EndRun(ERogueRunEndReason Reason);
```

Exec라서 콘솔에서 바로 실행 가능

->테스트 난이도를 낮추기 위해.

• 설정 변수(상태가 아니라 "설정값")

```
UPROPERTY(EditDefaultsOnly, Category = "Run|Maps")
FString LobbyMapPath = TEXT("/Game/Maps/L_Lobby");
```

```
UPROPERTY(EditDefaultsOnly, Category = "Run|Maps")
FString DefaultRunMapPath = TEXT("/Game/Maps/L_Run");

UPROPERTY(EditDefaultsOnly, Category = "Run|Stages")
TArray<FName> StageOrder;

UPROPERTY(EditDefaultsOnly, Category = "Run|Stages")
TMap<FName, FString> StageToMapPath;
```

LobbyMapPath: f런 종료 후 돌아 갈 맵.

DefaultRunMapPath: 스테이지 맵이 아직 없어도 런을 진행시키기 위한 기본 런 맵.

StageOrder: 논리적 스테이지 순서 (Stage_01->Stage_02->Stage_03)

StageToMapPath : StageId-> 실제 맵 경로 매핑 테이블.

동작설명(CPP)

생성자: 스테이지 기본값 구성.

```
StageOrder = { FName("Stage_01"), FName("Stage_02"), FName("Stage_03") };

StageToMapPath.Add(FName("Stage_01"), DefaultRunMapPath);
StageToMapPath.Add(FName("Stage_02"), DefaultRunMapPath);
StageToMapPath.Add(FName("Stage_03"), DefaultRunMapPath);
```

스테이지 이름 / 맵이 미정이라도 3스테이지 진행이라는 논리 흐름을 먼저 만들기 위함
□.

현재는 스테이지가 바뀌어도 이동 맵은 전부 L_Run

->나중에 스테이지 별 맵이 생기면 StageToMapPath만 수정하면 확장 가능.

- StartRun(): 런 시작점.

```
if (!EnsureServerAuth(TEXT("StartRun")))
{
    return;
}

ARogueGameState* GS = GetRogueGS();

if (!GS)
{
    return;
}

if (GS->GetRunActive())
{
    UE_LOG(LogTemp, Warning, TEXT("StartRun ignored: Run already active.));
    return;
}
```

```

}

GS->ResetRunState();
const FName FirstStage = (StageOrder.Num() > 0) ? StageOrder[0] : FName("Stage_01");

GS->SetStageld(FirstStage);
GS->SetEndReason(ERogueRunEndReason::None);
GS->SetRunActive(true);

ServerTravelToStage(FirstStage);

```

코드의 이유:

1. 서버 권위 가드

GameMode는 원래 서버에만 있지만, 습관적으로 가드 넣어 두면 디버깅 편해짐

2. 중복 방지

GS->GetRunActive()가 true이면 런이 이미 진행 중이므로 무시.

로그도 찍혀서 왜 실행이 안 됐는지 즉시 확인 가능.

3. 상태 초기화 -> 시작 상태 세팅

ResetRunState()로 이전 런 찌꺼기 제거.

Stageld, EndReason, RunActive를 시작 값으로 맞춤.

4. 마지막에 ServerTravel

상태를 먼저 세팅한 다음 트래블하는 순서라, 추후 UI/세이브 연동 시에도 안정적.

- AdvanceStage(): 스테이지 진행 진입점.

```

if (!EnsureServerAuth(TEXT("AdvaneStage")))
{
    return;
}

```

```

}

ARogueGameState* GS = GetRogueGS();

if (!GS)
{
    return;
}

if (!GS->GetRunActive())
{
    UE_LOG(LogTemp, Warning, TEXT("AdvanceStage ignored: Run is not active"));
    return;
}

const FName Current = GS->GetStageld();
const FName Next = GetNextStageld(Current);

if (Next.IsNone())
{
    EndRun(ERogueRunEndReason::BossKilled);
    return;
}

GS->SetStageld(Next);

ServerTravelToStage(Next);

```

런이 진행 중이 아닐 때 AdvanceStage가 들어오면 무시

->추후 "사망/보스 처치" 이벤트가 여러 번 들어오는 상황에서도 안전한 패턴

GetNextStageld()가 NAME_None을 반환하면 "마지막 스테이지 다음"

->현재는 이를 성공처리로 연결해 뼈대를 완성해 둔 상태.

EndRun(): 런 종료 진입점.

```
if (!EnsureServerAuth(TEXT("EndRun")))
{
    return;
}

ARogueGameState* GS = GetRogueGS();

if (!GS) return;

GS->SetRunActive(false);
GS->SetEndReason(Reason);

if (UWorld* World = GetWorld())
{
    World->ServerTravel(LobbyMapPath);
}
```

종료는 항상 단일 진입점(EndRun)로만 처리하는 구조

현재는 종료 시 로비로 이동만 하지만, 나중에 여기서 SaveSubSystem 트리거를 붙이면 저장 삭제도 추가할 수 있음.

유틸 함수들.

GetRogueGS()

```
return GetGameState<ARogueGameState>();
```

GameState를 매번 캐스팅 하지 않도록 도와주는 헬퍼.

GetNextStageId()

```
const int32 Index = StageOrder.IndexOfByKey(CurrentStageId);
if (Index == INDEX_NONE)
{
    return (StageOrder.Num() > 0) ? StageOrder[0] : NAME_None;
}

const int32 NextIndex = Index + 1;

return (StageOrder.IsValidIndex(NextIndex)) ? StageOrder[NextIndex] : NAME_None;
```

StageOrder에서 현재 스테이지의 인덱스를 찾고, 다음 인덱스를 반환

마지막이면 NAME_None를 반환.

ServerTravelToStage(StageId)

```
if (UWorld* World = GetWorld())
{
    //매핑 없으면 DefaultRunMapPath로 간다.
    const FString* Found = StageToMapPath.Find(StageId);
    const FString& Target = Found ? *Found : DefaultRunMapPath;

    World->ServerTravel(Target);
}
```

StageToMapPath에서 경로를 찾고 없으면 DefaultRunMapPath로 fallBack

World->ServerTravel(Target) 실행.

EnsureServerAuth(FuncName)

```
if (!HasAuthority())
{
    UE_LOG(LogTemp, Warning, TEXT("%s called on non-authority."), FuncName);
    return false;
}
return true;
```

HasAuthority가 아니면 경고 로그 + false 반환

서버 전용 흐름을 강제

테스트 방법:

콘솔에서 StartRun, AdvanceStage, AdvanceStage로 이어지는지 확인.